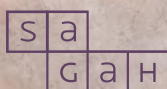


PENSAMENTO COMPUTACIONAL

Rubem Dutra Ribeiro Fagundes



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Comando condicional `if` (estruturas simples e compostas) — Python

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Definir os operadores lógicos e relacionais.
- Discutir os comandos `if` e `if... else`.
- Resolver problemas computacionais utilizando os comandos `if` e `if... else`.

Introdução

Como sabemos, o mundo está profundamente digitalizado. É inquestionável o fato de que os sistemas digitais estão constantemente tomando decisões, em frações de segundos. Uma importante parcela dessas decisões pode resultar em danos e acidentes terríveis, bem como na perda de vidas humanas.

A lógica de decisão adotada por uma máquina está registrada em seus circuitos de memória, ou seja, tal lógica foi previamente elaborada e programada na máquina. Em outras palavras, um ser humano, com inteligência — mais especificamente, um programador —, desenvolveu e implantou na máquina o processo de decisão e os procedimentos subsequentes, sendo ele, então, o responsável direto pelas ações da máquina. Afinal, máquinas não pensam, não raciocinam, não são inteligentes.

Assim, formalizar o processo de decisão que envolve qualquer tarefa que desejamos que a máquina realize constitui um importante e necessário conhecimento do programador. Este, então, utilizará operadores lógicos e instruções específicas para implantar suas decisões.

Neste capítulo, você vai estudar a forma de programar o processo de decisão na máquina, aprendendo sobre as estruturas lógicas que governam esse processo. Você também vai analisar em detalhes o comando

`if`, empregado na programação de computadores, e suas formas de utilização. Por fim, você vai verificar como resolver problemas utilizando a instrução `if` na linguagem de programação Python.

Definição dos operadores lógicos e relacionais

A essência de qualquer decisão é, em última análise, uma **comparação**. A todo momento, realizamos comparações dos mais variados tipos, envolvendo os mais diferentes assuntos, tópicos, ações, desejos, dentre outras necessidades. Podemos classificar nossos modos de comparação nas seguintes categorias, conforme aponta Law (2008):

- Equivalência ou similaridade: igual ou diferente.
- Qualidade: melhor ou pior.
- Dimensão: maior ou menor.
- Quantidade: mais ou menos, muito ou pouco.

Em programação, tais formas de comparação estarão presentes na comparação envolvida no **procedimento de decisão** que será programado. No entanto, observe que algumas das categorias de comparação são claramente subjetivas, isto é, exigem uma interpretação consciente e inteligente para serem realizadas. Por exemplo, quando dizemos que “é melhor beber suco de laranja sem açúcar do que suco de limão sem açúcar”, estamos estabelecendo a opinião do autor da sentença sobre a experiência dele em beber esses sucos.

No entanto, lidando com a realidade digital das máquinas, que não são inteligentes nem têm experiências de vida, não há contexto subjetivo ou experiência para a representação desse tipo de sentença. Desse modo, um aspecto importante da programação é que qualquer comparação envolve a quantificação de algum dado que caracterize a comparação necessária para a decisão.

Voltando ao exemplo, para programar de algum modo a decisão sobre o suco de laranja e o suco de limão, pode-se usar a medida de acidez, em função da quantidade percentual de ácido cítrico presente nos sucos. Então, uma forma de representar a capacidade de ingestão do suco sem açúcar na sentença “melhor beber suco de laranja do que suco de limão” é dizendo que “beber sem açúcar implica beber suco com baixa acidez, e a acidez do suco de laranja é menor do que a acidez do suco de limão”, conforme teorizam Russell e Norvig (2013).

Operadores relacionais

Para representar as sentenças da declaração da comparação, usualmente denominada **teste lógico** ou **condição lógica**, utilizam-se os operadores relacionais:

- **A = B, A igual a B**
- **A ≠ B, A diferente de B**
- **A > B, A maior do que B**
- **A ≥ B, A maior ou igual a B**
- **A < B, A menor do que B**
- **A ≤ B, A menor ou igual a B**

Em lógica, A e B podem ser quaisquer conceitos, abstratos ou não. Em programação, no entanto, A e B são **variáveis ou constantes** que serão avaliadas no momento em que a instrução de comparação for executada, conforme leciona Lutz (2011).

Em Python, bem como em outras conhecidas linguagens de programação, os operadores relacionais são representados no código conforme mostra o Quadro 1.

Quadro 1. Operadores relacionais

Operador relacional	Descrição	Representação em Python
A = B	A igual a B	A == B
A ≠ B	A diferente de B	A != B
A > B	A maior do que B	A > B
A ≥ B	A maior ou igual a B	A >= B
A < B	A menor do que B	A < B
A ≤ B	A menor ou igual a B	A <= B

O operador relacional será utilizado para representar quaisquer condições de decisão, sendo a forma como será transcrita em código a condição lógica de decisão de qualquer enunciado. Considere o seguinte exemplo:

Sentença: A idade mínima para entrada no cinema é 18 anos.

Representação: PERMISSÃO PARA INGRESSO: $IDADE \geq 18$

Note que tal condição lógica poderia fazer parte, por exemplo, da tomada de decisão em um sistema automatizado de acesso a uma sala de cinema. Assim, para esse exemplo, suponha um sistema que obtém a idade por meio da leitura da carteira de estudante ou de outra forma de identificação. Um fragmento do algoritmo referente à decisão seria:

LER IDADE DO ESTUDANTE

SE A $IDADE \geq 18$ ANOS:

ENTÃO

 ACESSO PERMITIDO (ACIONAR ABERTURA DE PORTA DE
 ACESSO E MENSAGEM “ACESSO PERMITIDO”)

SENÃO

 ACESSO NEGADO (FECHAR OU MANTER FECHADA A
 PORTA DE ACESSO E MENSAGEM “IDADE MÍNIMA NÃO
 ATENDIDA — ACESSO NEGADO”)

Note que a comparação está na condição lógica $IDADE \geq 18$ ANOS, e o resultado da comparação leva à decisão de permitir ou não o acesso à sala de cinema.

Operadores lógicos

Para a construção de testes lógicos mais completos, envolvendo mais de uma condição comparativa, utilizaremos os operadores lógicos OU, E e NÃO (NEGADO ou NEGAÇÃO). Assim, tais operadores lógicos servirão como conectores entre condições comparativas, representando de forma completa uma dada sentença de decisão, conforme lecionam Russell e Norvig (2013). A seguir, serão abordados os operadores lógicos OU, E e NEGAÇÃO.

Operador OU (A ou B)

Esse operador é muito utilizado em lógica e constitui a operação que conecta duas condições, A e B, que podem ocorrer **conjuntamente ou de forma alternada**. Quando se diz A **OU** B, significa dizer que:

- A condição A e a condição B podem ambas ser verdadeiras.
- A pode ser verdadeira e B falsa.
- A pode ser falsa e B verdadeira.

Vejamos um exemplo:

Sentença: O animal bovino possui cores branca ou preta.

Representação: BOVINOS: (Cor = BRANCA) OU (Cor = PRETA)

No exemplo, um BOVINO poderá ser de cor BRANCA, poderá ser de cor PRETA e poderá ser de duas cores, BRANCA e PRETA (bovinos malhados). No Quadro 2, podemos verificar a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como condição VERDADEIRA e F como condição FALSA.

Quadro 2. Tabela verdade para o operador lógico OU

A	B	A ou B
F	F	F
F	V	V
V	F	V
V	V	V

O operador A ou B somente é falso (o resultado é F) quando **A é falso e B é falso**.

Operador E (A e B)

Trata-se do operador lógico que relaciona duas condições, A e B, que ocorrem **somente juntas**. Quando se diz A E B, significa dizer que A acontece (é verdadeiro) e B também necessariamente acontece (é verdadeiro). Vejamos um exemplo:

Sentença: Toda autorização de pagamento deve ser assinada pelo presidente e pelo tesoureiro.

Representação: AUTORIZAR PAGAMENTO: (ASSINATURA DO PRESIDENTE = SIM) E (ASSINATURA DO TESOUREIRO = SIM)

No exemplo, o presidente e o tesoureiro devem ambos assinar a autorização; isto é, as duas condições devem ser satisfeitas. Se somente um deles, isoladamente, assinar o documento, sem a assinatura do outro, não haverá autorização.

O Quadro 3 apresenta a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.

Quadro 3. Tabela verdade para o operador lógico E

A	B	A e B
F	F	F
F	V	F
V	F	F
V	V	V

O operador A · B somente é verdadeiro (o resultado é V) quando A é verdadeiro e B é verdadeiro.

Operador de negação (não A)

Este é o operador mais simples utilizado em lógica e consiste na operação lógica de inversão do estado lógico da condição. Quando se diz não A, ou a negação de A, significa dizer que, se A existe ou é verdadeiro, não A inexistiu ou é falso. Vejamos um exemplo:

Sentença: Acionar a admissão de água na caixa quando ela estiver vazia.

Representação: Admissão de água caixa: (não CHEIA)

No exemplo, a caixa d'água deverá receber água quando estiver vazia, isto é, a condição contrária ou negada de cheia. No Quadro 4, pode-se conferir a tabela verdade do operador, representando todas as situações em que o operador é verdadeiro ou falso. Considere V como fato VERDADEIRO e F como fato FALSO.

Quadro 4. Tabela verdade para o operador lógico de negação (não A)

A	não A
F	V
V	F

O Quadro 5 resume os operadores lógicos e suas representações e o respectivo código de programa da linguagem Python.

Quadro 5. Operadores lógicos e sua representação em Python

Operador lógico	Descrição	Representação em Python
A ou B	A OU B	A or B
A e B	A E B	A and B
não A	NÃO A	not A

Segundo Lutz (2011), utilizando os operadores relacionais e os operadores lógicos, pode-se representar testes lógicos complexos, envolvendo, em um único teste, situações com mais de uma condição relacional, conectadas por meio de operadores lógicos. Vejamos um exemplo:

Sentença: Em um dado país, os homens devem se apresentar ao serviço militar aos 18 anos, enquanto as mulheres podem se apresentar com idade maior ou igual a 18 anos e menor do que 36 anos.

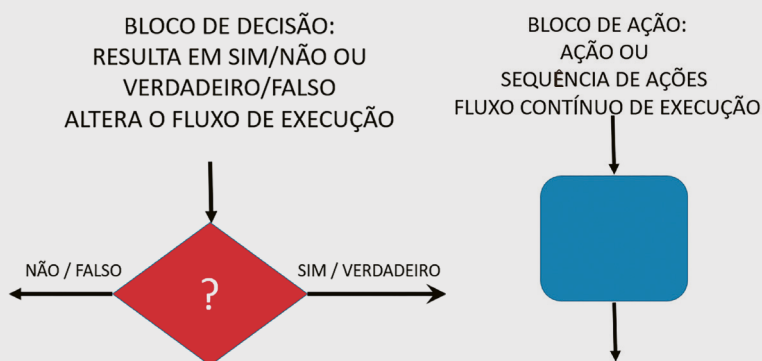
Representação: SERVIÇO MILITAR: (HOMENS: IDADE = 18 ANOS) ou (MULHERES: IDADE $\geq$ 18 ANOS e IDADE $<$ 36 ANOS)

Na próxima seção, vamos estudar e utilizar a instrução `if` para a programação de decisões em linguagem Python.



Fique atento

As representações gráficas de algoritmos e programas são amplamente utilizadas, tornando mais compreensível o comportamento funcional. O fluxograma possui símbolos que representam diferentes tipos de instruções. Os símbolos mais utilizados, inclusive fora da área de programação (sendo empregados na apresentação de protocolos de operação, protocolos de atendimento, protocolos de emergência, dentre outros), são os seguintes:



Analizando os comandos `if` e `if...else`

O **comando condicional** `if`, do inglês “se”, é encontrado em praticamente todas as linguagens de programação de alto nível, sendo talvez um dos mais importantes e, sem dúvida, um dos mais antigos comandos existentes no conjunto de instruções dos primeiros computadores.

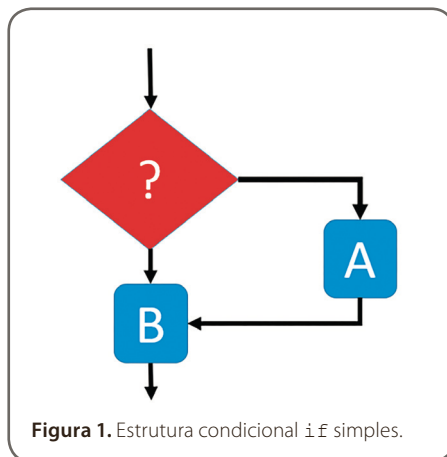
Estrutura condicional `if` simples

A instrução `if` é a instrução básica em programação para a construção de decisões que a máquina vai realizar durante a execução do programa. Observe que o processo de decisão altera a sequência das instruções que serão realizadas pela máquina, também chamada de **fluxo de processamento**.

Veja o formato em Python da instrução `if` simples:

```
if condição lógica:
    #Bloco A
    Bloco de comandos executados caso a condição
    lógica seja verdadeira
#Bloco B
Comandos executados após o teste
```

Observe que, em Python, a tabulação e o alinhamento das instruções definem os blocos de cada parte do comando `if`. Assim, se a `condição lógica` for VERDADEIRA, o `Bloco A` de comandos será executado, e, após o término das instruções que compõem o Bloco A, o Bloco B será executado. Se, no entanto, a condição lógica for FALSA, somente o Bloco B de instruções será executado. A Figura 1 a seguir exibe um diagrama com a representação gráfica dessa situação.



Vejamos um exemplo:

```
if temperatura>25:
    print("Ambiente quente, ajustando ar-condicionado
    para clima de verão")
    AR_CONDICIONADO(VERAO)
print("Ligando o ar-condicionado")
AR_CONDICIONADO(ON)
```

No exemplo, o ar-condicionado será sempre acionado com a mensagem ao usuário: “Ligando o ar-condicionado”; porém, será ajustado para a função verão **somente se a temperatura for maior do que 25°**, avisando o usuário por meio da mensagem: “Ambiente quente, ajustando ar-condicionado para clima de verão”.

Estrutura condicional `if... else`

A estrutura `if... else` (do inglês “se... senão”) é um legado da programação estruturada, pois orienta o fluxo da informação em um sentido “*top-down*”, isto é, de cima para baixo, desde o início até o fim do código. Esse tipo de estrutura atende à situação em que é necessário escolher uma de duas opções, de acordo com o resultado da comparação. Veja abaixo a sintaxe da instrução `if... else`:

```
if condição lógica:
    #Bloco A
    Bloco de comandos executados caso a condição
    lógica seja VERDADEIRA
else:
    #Bloco B
    Bloco de comandos executados caso a condição
    lógica seja FALSA
#Bloco C
Comandos executados após o teste.
```

Se a **condição lógica** for VERDADEIRA, o **Bloco A** de comandos será executado. Se, no entanto, a **condição lógica** for FALSA, somente o **Bloco B** de instruções será executado. Após o término da execução das instruções do **Bloco A** ou do **Bloco B**, as instruções do Bloco C serão executadas. A Figura 2 a seguir exibe um diagrama com a representação gráfica dessa situação.

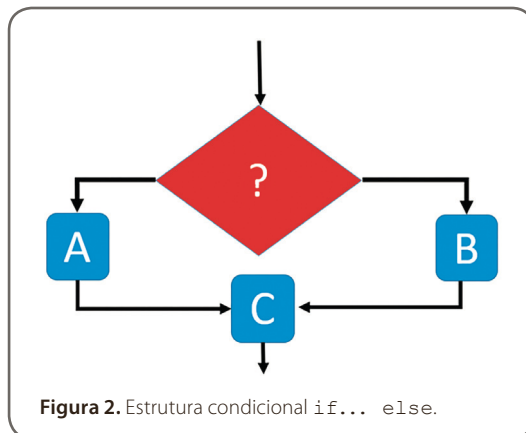


Figura 2. Estrutura condicional `if... else`.

Vejamos um exemplo:

```
if temperatura>25:
    print("Ambiente quente, ajustando ar-condicionado
    para clima de verão")
    AR _CONDICIONADO(VERAO)
else :
    print("Ambiente frio, ajustando ar-condicionado
    para clima de inverno")
    AR _CONDICIONADO(INVERNO)
print("Ligando o ar-condicionado")
AR _CONDICIONADO(ON)
```

No exemplo, o ar-condicionado será sempre acionado, com a mensagem ao usuário “Ligando o ar-condicionado”. Será ajustado para o verão, avisando o usuário por meio da mensagem “Ambiente quente, ajustando ar-condicionado para clima de verão”, **se a temperatura for maior do que 25°**, ou será ajustado para o inverno, avisando o usuário por meio da mensagem “Ambiente frio, ajustando ar-condicionado para clima de inverno”, **se a temperatura for menor ou igual a 25°**.

Estrutura condicional `if... elif... else`

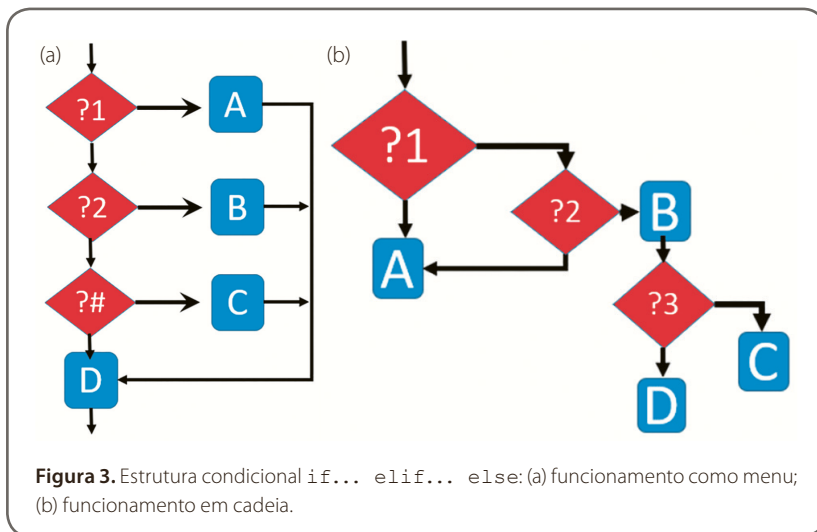
Essa estrutura permite o encadeamento de vários testes, selecionando conjuntos diferentes de ações, de acordo com o resultado em cascata de cada condição lógica. Em Python, além de proporcionar a geração de um sistema de decisão mais complexo, a estrutura `if... elif... else` é frequentemente usada no menu de opções para o usuário. Veja abaixo a sintaxe da instrução `if... elif... else`:

```
if condição lógica1:
    #Bloco A
    Bloco de comandos executados caso a condição
    lógica1 seja VERDADEIRA
elif condição lógica2:
    #Bloco B
    Bloco de comandos executados caso a condição
    lógica1 seja FALSA e a condição lógica2 seja
    VERDADEIRA
elif condição lógica3:
    #Bloco C
    Bloco de comandos executados caso a condição ló-
    gica1 seja FALSA, a condição lógica2 seja FALSA
    e a condição lógica3 seja VERDADEIRA
    ...
else:
    #Bloco N
    Bloco de comandos executados caso NENHUMA DAS
    condições lógicas sejam VERDADEIRAS

#Bloco X
Comandos executados após o teste.
```

Nessa estrutura, se a **condição lógica1** for VERDADEIRA, o **Bloco A** de comandos será executado. Se, no entanto, a **condição lógica1** for FALSA, a **condição lógica2** será avaliada e, se for VERDADEIRA, somente o **Bloco B** de instruções será executado. Caso as condições lógicas 1 e 2 forem ambas falsas, a condição lógica3 será avaliada e, se for VERDADEIRA, somente o **Bloco C** de instruções será executado, e assim sucessivamente, para cada novo teste em cadeia. Por último, se nenhuma condição lógica dos testes em cadeia for VERDADEIRA, então, o Bloco N de instruções será executado. Após o término da execução de algum dos blocos de instruções dos testes do sistema de decisão, as instruções do Bloco X serão executadas.

A Figura 3 exibe um diagrama com a representação gráfica dessa situação.



Link

Nos exemplos apresentados, a função `AR _CONDICIONADO`(parâmetro) foi ilustrativa para mostrar o uso de funções de programação em Python. Python possui um amplo conjunto de funções prontas, denominadas pacotes, para uma grande variedade de usos, como criação de jogos, manipulação de dados, geração de gráficos, processamento de imagens, dentre outros projetos. Saiba mais sobre esses excelentes recursos acessando o [link](https://qrgo.page.link/38eFU) a seguir.

<https://qrgo.page.link/38eFU>

Resolução de problemas computacionais utilizando os comandos `if` e `if... else`

A partir dos conhecimentos sobre a sintaxe e os usos das diferentes estruturas de decisão do comando `if`, nesta seção, vamos analisar e resolver alguns problemas. Veja o enunciado a seguir:

O plano diretor de desenvolvimento urbano de uma cidade determina qual é o percentual de área máximo destinado para garagem em relação à área total do terreno da casa, dependendo da localização desse terreno na cidade:

- Para a zona norte da cidade, o percentual máximo é de 25%.
- Para as zonas leste e oeste da cidade, o percentual máximo é de 30%.
- Para a zona sul, menos povoada, o percentual máximo é de 40%.

Uma empresa de arquitetura está com vários contratos e necessita calcular rapidamente esse percentual, antes de iniciar os projetos. Faça um programa que recebe as medidas do terreno e da garagem e a zona onde estará localizado o imóvel, calcula o percentual de ocupação da área da garagem em relação ao terreno e emite mensagem sobre o atendimento às regras de ocupação conforme o plano diretor.

Desenvolvimento do código

Pelo enunciado, existem cinco variáveis, a saber:

```
largura da garagem em metros
profundidade da garagem em metros
largura do terreno em metros
profundidade do terreno em metros
zona de localização do terreno
```


Com essas variáveis, deve-se executar as seguintes tarefas:

Calcular área do terreno = largura do terreno * profundidade do terreno

Calcular área da garagem = largura da garagem * profundidade da garagem

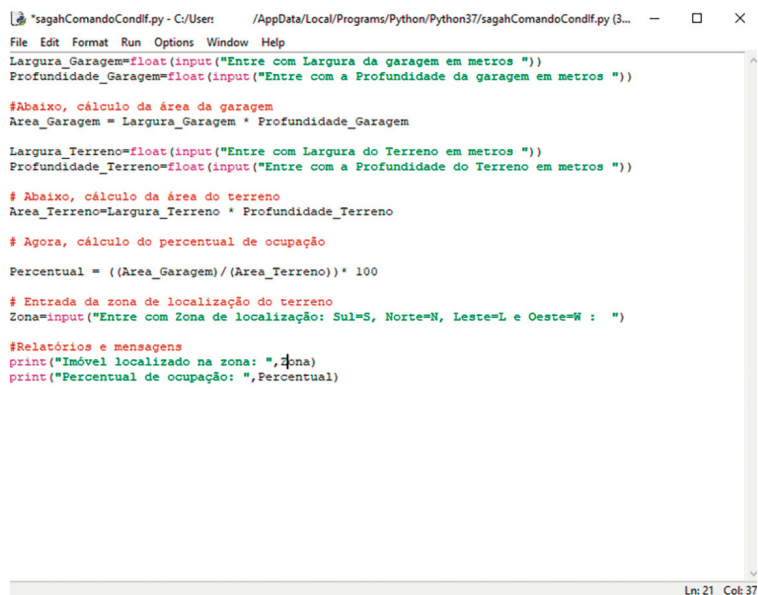
Calcular percentual de ocupação = (área da garagem / área do terreno) * 100

Apresentar em tela o resultado do percentual de ocupação

Comparar o percentual com os percentuais definidos por cada zona

Emitir mensagem sobre a situação – atendimento ou não atendimento das regras de zoneamento do plano diretor

Para tornar claro o código, vamos inicialmente programar a entrada de dados e os cálculos que serão executados, conforme apresentado na Figura 4.



```
File Edit Format Run Options Window Help
Largura_Garagem=float(input("Entre com Largura da garagem em metros "))
Profundidade_Garagem=float(input("Entre com a Profundidade da garagem em metros "))

#Abaixo, cálculo da área da garagem
Area_Garagem = Largura_Garagem * Profundidade_Garagem

Largura_Terreno=float(input("Entre com Largura do Terreno em metros "))
Profundidade_Terreno=float(input("Entre com a Profundidade do Terreno em metros "))

# Abaixo, cálculo da área do terreno
Area_Terreno=Largura_Terreno * Profundidade_Terreno

# Agora, cálculo do percentual de ocupação
Percentual = ((Area_Garagem)/(Area_Terreno)) * 100

# Entrada da zona de localização do terreno
Zona=input("Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : ")

#Relatórios e mensagens
print("Imóvel localizado na zona: ",Zona)
print("Percentual de ocupação: ",Percentual)
```

Ln: 21 Col: 37

Figura 4. Entrada de dados do programa exemplo.

Então, vamos programar a sequência de testes correspondente ao zoneamento, conforme as regras do plano diretor. Observe que, conforme a zona, deve-se testar o percentual de ocupação e comparar com o percentual permitido por zona, emitindo a mensagem de situação de atendimento às normas do plano diretor. Veja o fragmento de código mostrado na Figura 5.

```
if Zona=="N" and Percentual<=25:
    print("projeto atende norma de zoneamento do plano diretor")

elif Zona=="L" or Zona=="W" and Percentual<=30:
    print("projeto atende norma de zoneamento do plano diretor")

elif Zona=="S" and Percentual<=40:
    print("projeto atende norma de zoneamento do plano diretor")

else:
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Figura 5. Sequência de testes do programa exemplo.

Veja que, nos testes, foram utilizados os operadores lógicos para conectar as condições lógicas referentes à zona e ao percentual de ocupação. Veja na Figura 6 o código completo.

```
File Edit Format Run Options Window Help
#Abaixo, cálculo da área da garagem
Area_Garagem = Largura_Garagem * Profundidade_Garagem

Largura_Terreno=float(input("Entre com Largura do Terreno em metros "))
Profundidade_Terreno=float(input("Entre com a Profundidade do Terreno em metros "))

# Abaixo, cálculo da área do terreno
Area_Terreno=Largura_Terreno * Profundidade_Terreno

# Agora, cálculo do percentual de ocupação
Percentual = ((Area_Garagem)/(Area_Terreno))* 100

# Entrada da zona de localização do terreno
Zona=input("Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : ")

#Relatórios e mensagens
print("Imóvel localizado na zona: ",Zona)
print("Percentual de ocupação: ",Percentual)
# Sequencia de decisão

if Zona=="N" and Percentual<=25:
    print("projeto atende norma de zoneamento do plano diretor")

elif Zona=="L" or Zona=="W" and Percentual<=30:
    print("projeto atende norma de zoneamento do plano diretor")

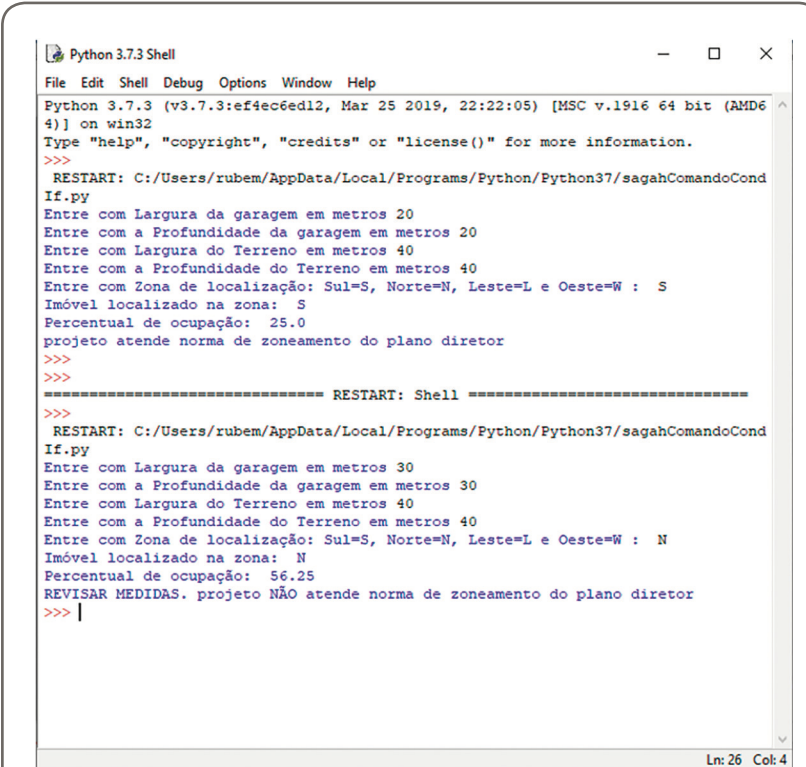
elif Zona=="S" and Percentual<=40:
    print("projeto atende norma de zoneamento do plano diretor")

else:
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Ln: 17 Col: 43

Figura 6. Código completo do programa exemplo.

A Figura 7 exibe a execução do programa.



```
Python 3.7.3 Shell
File Edit Shell Debug Options Window Help
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 22:22:05) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/rubem/AppData/Local/Programs/Python/Python37/sagahComandoCond
If.py
Entre com Largura da garagem em metros 20
Entre com a Profundidade da garagem em metros 20
Entre com Largura do Terreno em metros 40
Entre com a Profundidade do Terreno em metros 40
Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : S
Imóvel localizado na zona: S
Percentual de ocupação: 25.0
projeto atende norma de zoneamento do plano diretor
>>>
===== RESTART: Shell =====
>>>
RESTART: C:/Users/rubem/AppData/Local/Programs/Python/Python37/sagahComandoCond
If.py
Entre com Largura da garagem em metros 30
Entre com a Profundidade da garagem em metros 30
Entre com Largura do Terreno em metros 40
Entre com a Profundidade do Terreno em metros 40
Entre com Zona de localização: Sul=S, Norte=N, Leste=L e Oeste=W : N
Imóvel localizado na zona: N
Percentual de ocupação: 56.25
REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor
>>> |
```

Ln: 26 Col: 4

Figura 7. Execução do programa exemplo mostrado na Figura 6 em duas situações.

Existem formas diferentes de utilização da instrução `if`, combinando estruturas simples e complexas. Veja na Figura 8 a solução do problema anterior com nova sequência de decisão, que produz o mesmo resultado.

```
# Nova Sequencia de decisão
if Zona=="N":
    if Percentual<=25:
        print("projeto atende norma de zoneamento do plano diretor")

elif Zona=="L" or Zona=="W":
    if Percentual<=30:
        print("projeto atende norma de zoneamento do plano diretor")

elif Zona=="S":
    if Percentual<=40:
        print("projeto atende norma de zoneamento do plano diretor")

else:
    print("REVISAR MEDIDAS. projeto NÃO atende norma de zoneamento do plano diretor")
```

Figura 8. Versão alternativa para o segmento de testes do programa exemplo.

Como vimos, as estruturas de programação `if`, `if... else` e `if... elif... else` cobrem todas as formas e os diferentes modos de decisão. Assim, a principal tarefa do programador é a correta definição da lógica necessária para o processo de decisão, bem como das variáveis envolvidas para orientar essa decisão. Nesse sentido, o programador deve desenvolver plenamente o raciocínio lógico, saber analisar criticamente as condições que envolvem as decisões e explorar a linguagem de programação Python, criando novas formas para a resolução de problemas.



Referências

- LAW, S. *Filosofia*: guia ilustrado Zahar. 2. ed. Rio de Janeiro: Zahar, 2008. 352 p.
- LUTZ, M. *Programming Python*. 4. ed. Sebastopol: O'Reilly, 2011. 1585 p.
- RUSSELL, S.; NORVIG, P. *Inteligência artificial*. 3. ed. Rio de Janeiro: Elsevier, 2013. 1056 p.

Leituras recomendadas

- CORMEN, T. H. *Desmistificando algoritmos*. Rio de Janeiro: Elsevier; Campus, 2014. 188 p.
- CORMEN, T. H. *et al. Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier; Campus, 2012. 926 p.
- LUTZ, M. *Python pocket reference*: Python in your pocket. 5. ed. Sebastopol: O'Reilly, 2014. 254 p.
- MARUCH, S.; MARUCH, A. *Python for dummies*. New York: John Wiley & Sons, 2006. 432 p.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS